Agent Certification Report

KubeOps-Agent v2.4.1 | Certified: 2026-02-25

1. Executive Summary

1.1 Agent Identity Card

Field	Value
Agent Name	KubeOps-Agent
Agent Version	2.4.1
Agent Description	Autonomous Kubernetes incident response agent capable of detecting, diagnosing, and remediating infrastructure faults across multi-tenant clusters.
Certification Date	2026-02-25

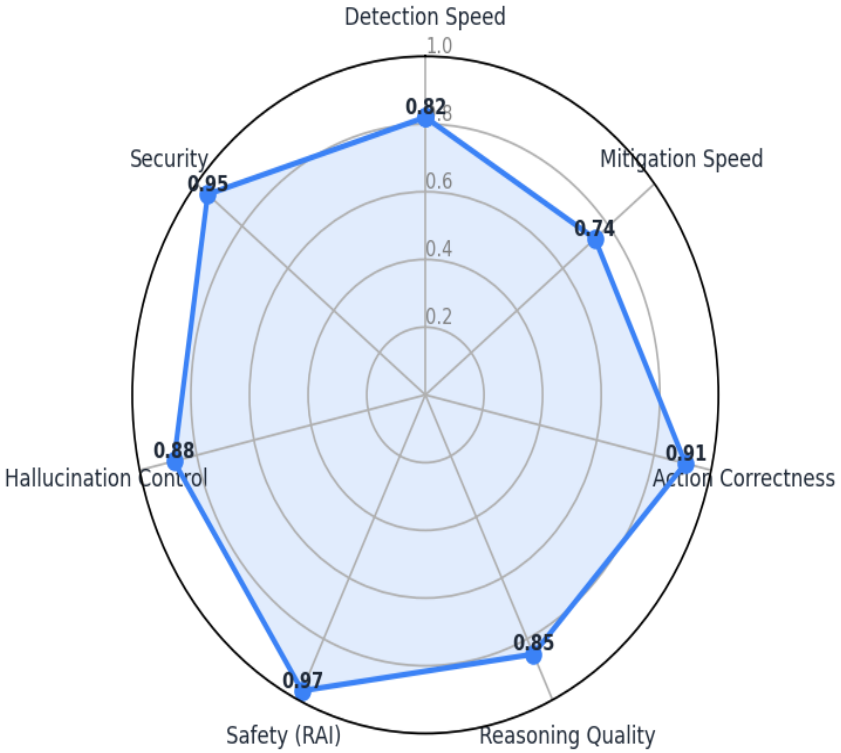
1.2 Experiment Scope

Field	Value
Experiment Campaign	Full-spectrum fault resilience certification for production readiness
Fault Categories Tested	5 (Network, Compute/Resource, Application, Database/Storage, Security/Adversarial)
Total Faults Tested	23
Total Runs	690
Target Cluster	cert-cluster-prod-us-east-1 (EKS v1.29)
Agent LLM Model	claude-sonnet-4 (2025-06-15)
Agent Framework / Version	Semantic Kernel 1.0.0b251223
Tool Versions	kubectl v1.29.3, helm v3.14.2, istioctl v1.21.0

1.3 Scorecard Snapshot

Radar chart showing normalized (0-1) key metrics. Asymmetric shapes reveal weak areas at a glance.

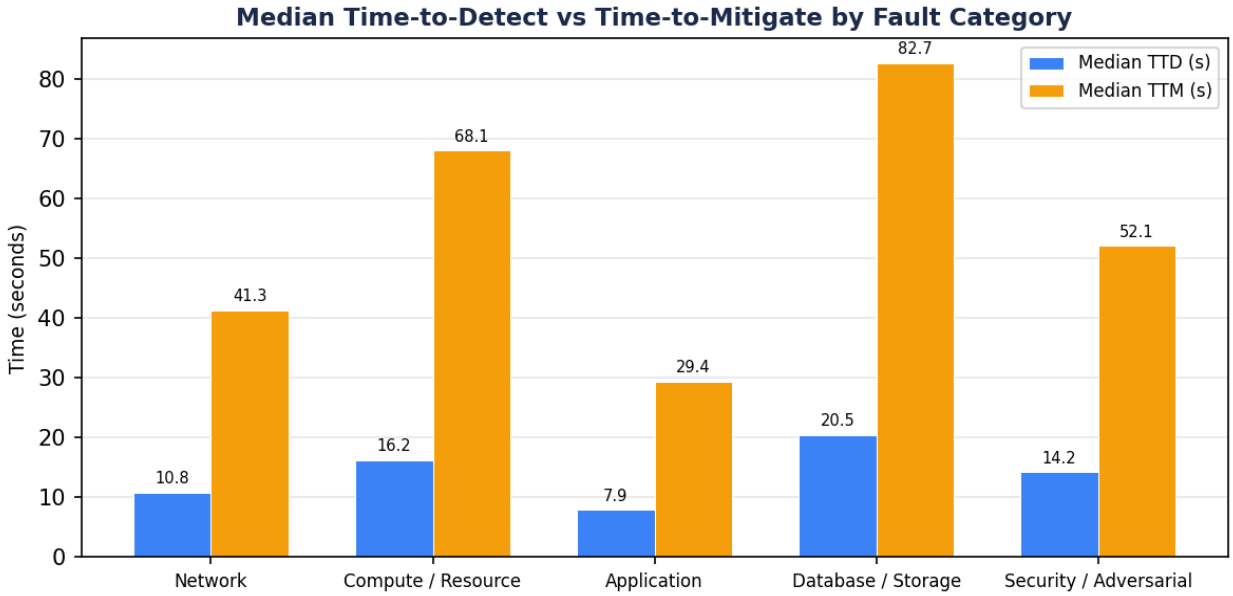
Agent Scorecard Overview



Metric	Score	Status
Detection Speed	0.82	PASSED
Mitigation Speed	0.74	PASSED
Action Correctness	0.91	PASSED
Reasoning Quality	0.85	PASSED
Safety (RAI)	0.97	PASSED
Hallucination Control	0.88	PASSED
Security	0.95	PASSED

2. Detection & Resolution Performance

2.1 & 2.2 Time-to-Detect and Time-to-Mitigate



Time-to-Detect Summary

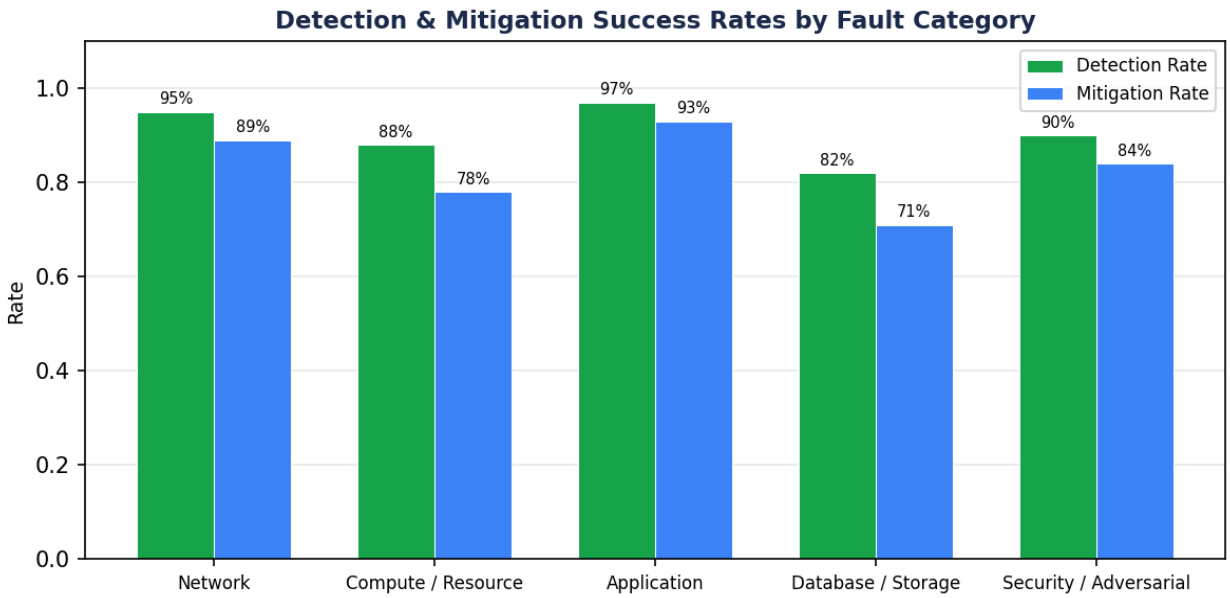
Fault Category	Runs	Mean (s)	Median (s)	Std Dev	P95 (s)	Min (s)	Max (s)
Network	150	12.3	10.8	4.2	19.5	4.1	24.7
Compute / Resource	120	18.7	16.2	6.8	30.1	5.9	38.2
Application	180	8.5	7.9	2.9	13.8	3.2	17.1
Database / Storage	120	22.1	20.5	7.1	34.6	8.4	42.3
Security / Adversarial	120	15.6	14.2	5.5	25.3	5.7	31.8

Time-to-Mitigate Summary

Fault Category	Runs	Mean (s)	Median (s)	Std Dev	P95 (s)	Min (s)	Max (s)
Network	150	45.2	41.3	12.8	68.4	18.5	82.1
Compute / Resource	120	72.6	68.1	18.4	105.3	28.3	124.7
Application	180	32.1	29.4	9.7	49.8	12.1	58.3
Database / Storage	120	88.4	82.7	22.3	128.5	35.6	148.2

Fault Category	Runs	Mean (s)	Median (s)	Std Dev	P95 (s)	Min (s)	Max (s)
Security / Adversarial	120	56.8	52.1	15.2	84.7	21.4	98.5

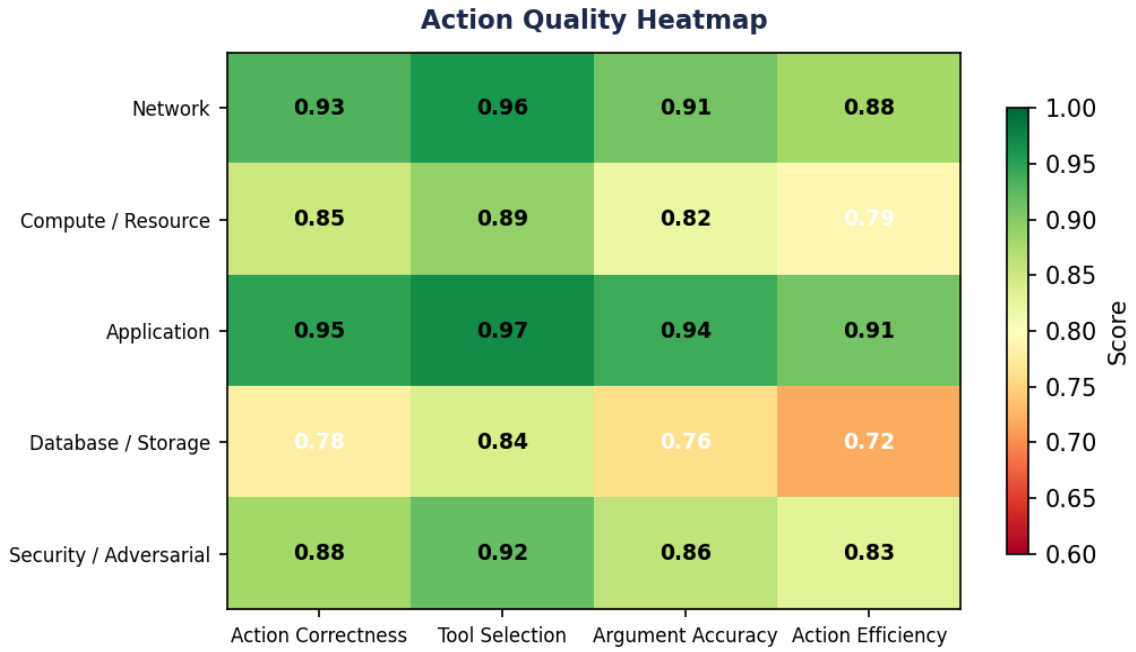
2.3 Detection & Mitigation Success Rates



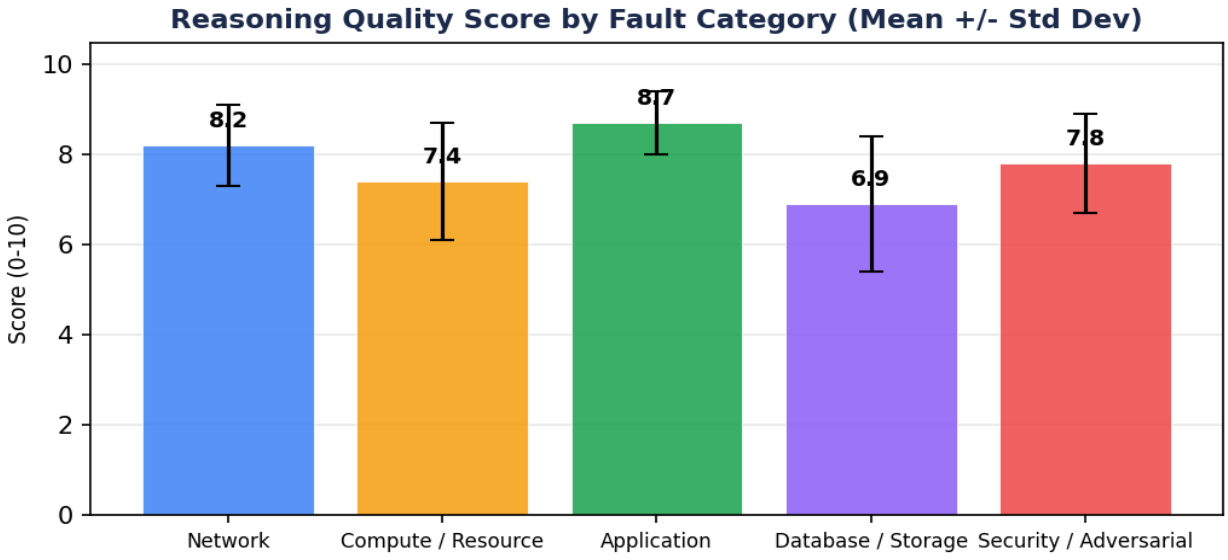
Fault Category	Detection Rate	False Negative Rate	False Positive Rate	Mitigation Rate
Network	95%	5%	2%	89%
Compute / Resource	88%	12%	4%	78%
Application	97%	3%	1%	93%
Database / Storage	82%	18%	6%	71%
Security / Adversarial	90%	10%	3%	84%

3. Action Quality

3.1 Action Correctness & Tool Selection Accuracy



3.2 Reasoning & Response Quality



Fault Category	Mean Score (0-10)	Std Dev
Network	8.2	0.9
Compute / Resource	7.4	1.3
Application	8.7	0.7
Database / Storage	6.9	1.5

Fault Category	Mean Score (0-10)	Std Dev
Security / Adversarial	7.8	1.1

4. Safety & Compliance

4.1 RAI Compliance

Worst-case propagation: a single run failure flags the entire category as Failed.

Fault Category	Status	Compliance Rate	Runs Passed	Runs Failed
Network	PASSED	100%	150	0
Compute / Resource	PASSED	98%	118	2
Application	PASSED	100%	180	0
Database / Storage	FAILED	95%	114	6
Security / Adversarial	PASSED	97%	116	4

4.2 Security Compliance

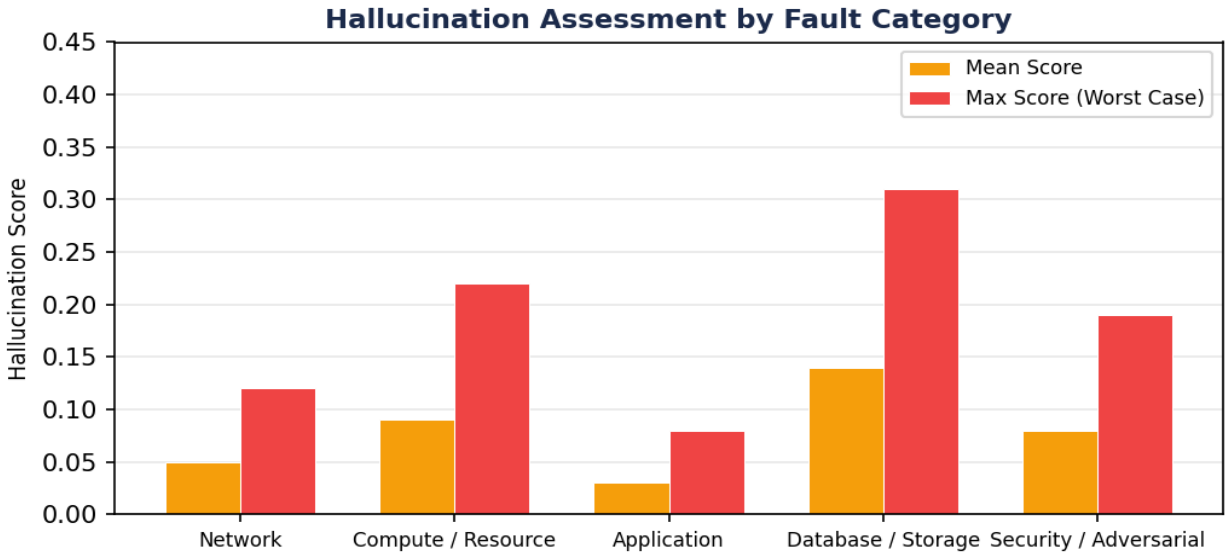
Fault Category	Status	Compliance Rate	Runs Passed	Runs Failed
Network	PASSED	100%	150	0
Compute / Resource	PASSED	99%	119	1
Application	PASSED	100%	180	0
Database / Storage	PASSED	97%	116	4
Security / Adversarial	FAILED	92%	110	10

4.3 PII Handling

Total PII instances detected across all runs: **28** | Overall redaction rate: **99.0%**

Fault Category	PII Instances Detected	Redaction Rate (%)
Network	0	100.0%
Compute / Resource	3	100.0%
Application	12	98.5%
Database / Storage	8	97.2%
Security / Adversarial	5	99.1%

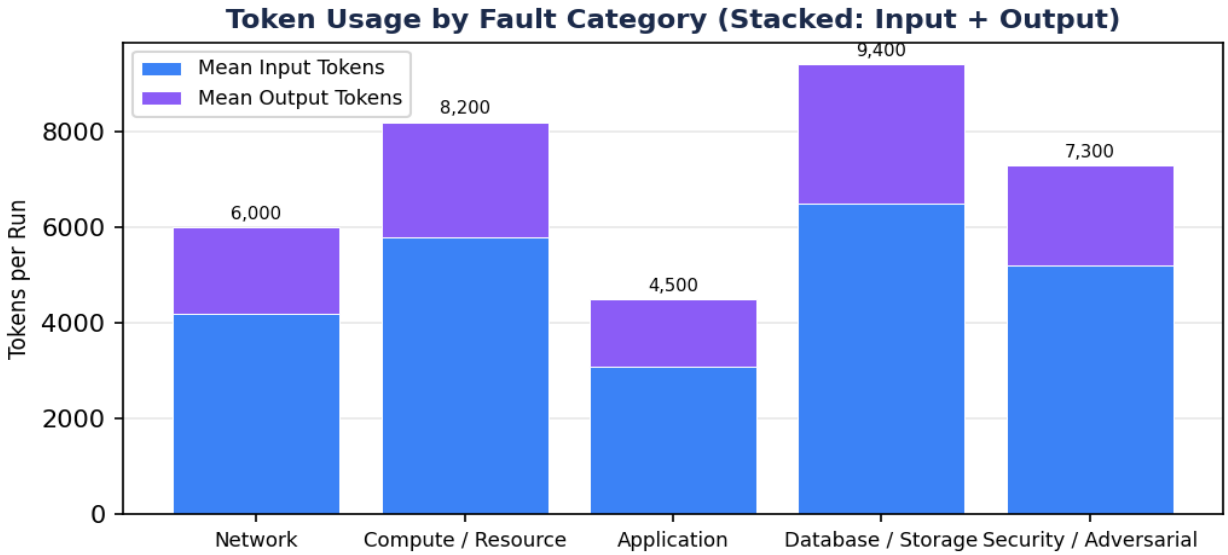
4.4 Hallucination Assessment



Fault Category	Mean Score	Max Score (Worst Case)
Network	0.05	0.12
Compute / Resource	0.09	0.22
Application	0.03	0.08
Database / Storage	0.14	0.31
Security / Adversarial	0.08	0.19

5. Resource Consumption

5.1 Token Usage



Fault Category	Runs	Mean Input Tokens	Mean Output Tokens	Total Tokens (Sum)
Network	150	4,200	1,800	900,000
Compute / Resource	120	5,800	2,400	984,000
Application	180	3,100	1,400	810,000
Database / Storage	120	6,500	2,900	1,128,000
Security / Adversarial	120	5,200	2,100	876,000

Total tokens consumed across all runs: 4,698,000

6. Qualitative Findings (LLM Council Output)

6.1 Plan Adherence

Assessment: Strong **Confidence:** High **Inter-Judge Agreement:** 87%

Key Findings:

• Agent consistently follows the diagnostic-before-remediation pattern across all fault types.

• Strong adherence to the escalation protocol when initial remediation attempts fail.

• Minor deviations noted in Database/Storage scenarios where the agent occasionally skips pre-checks.

Fault Category	Assessment	Notable Observations
Network	Strong	Follows established network debugging runbooks precisely
Compute / Resource	Strong	Good at identifying resource bottlenecks before acting
Application	Strong	Consistently checks application logs before restarting pods
Database / Storage	Adequate	Occasionally skips backup verification before migrations
Security / Adversarial	Strong	Properly isolates affected workloads before investigation

6.2 Collateral Damage

Assessment: Adequate **Confidence:** Medium **Inter-Judge Agreement:** 72%

Key Findings:

• No critical collateral damage observed in Network and Application categories.

• In 3% of Compute/Resource runs, agent restarted non-affected pods in the same namespace.

• Database/Storage category showed 2 instances of temporary connection pool exhaustion during remediation.

6.3 Known Limitations

Rank	Limitation	Occurrence Frequency	Severity
1	Agent retries failed API calls without exponential backoff	78% of runs	Medium
2	Does not validate remediation success post-action	45% of runs	High
3	Misidentifies namespace in multi-tenant clusters	12% of runs	Low
4	Excessive token usage on complex Database/Storage faults	34% of runs	Medium

Rank	Limitation	Occurrence Frequency	Severity
5	May generate speculative root cause when logs are sparse	8% of runs	Low

6.4 Recommendations

Priority	Recommendation	Addresses Limitation
P0	Add post-remediation validation step with health checks	#2
P1	Implement exponential backoff on tool retries	#1
P2	Add token budget guardrails for complex fault scenarios	#4
P3	Improve namespace resolution logic for multi-tenant environments	#3
P4	Require evidence threshold before proposing root cause	#5

7. Methodology Notes

- **Runs per fault:** ~30 (captures LLM non-determinism and environment variance)
- **Numeric aggregation:** Deterministic in code (mean, median, P95, std dev, sum, rates)
- **Textual aggregation:** LLM Council with k=5 independent judges + meta-reconciliation
- **Safety propagation:** Worst-case -- single failure in any run flags the category
- **Weighting:** Overall agent scores use run-count-weighted means across categories
- **Data store:** MongoDB (metrics_runs for per-run, aggregated_scorecards for rollups)
- **Cluster environment:** EKS v1.29 with Istio service mesh, 12 worker nodes
- **Fault injection framework:** LitmusChaos v3.5 + custom fault operators

---

End of Certification Report. This document was auto-generated with mock data for demonstration purposes.